

WARCHEST

Internal Technical Build Plan

1 Scope

Five components, built and audited in this order: (1) standard ERC20 token, (2) custom Uniswap v4 fee hook, (3) governance contract (levels + voting), (4) treasury vault contract, (5) off-chain level indexer + Hyperliquid keeper bot. Contracts 3 and 4 are strictly separate: the vault only executes pre-validated orders, the governance contract only computes levels and tallies votes. No contract talks to Hyperliquid directly; only the keeper does.

2 Contract Architecture

Component	Responsibility
<code>WarchestToken.sol</code>	Plain ERC20. No transfer-tax logic, no fee logic of any kind.
<code>WarchestHook.sol</code>	Uniswap v4 hook attached to the TOKEN/ETH pool. Reads swap direction (buy/sell), takes the 10% fee at swap time, forwards it to the treasury vault. Mirrors the pattern already proven in production by Stakd on Robinhood Chain.
<code>WarchestGovernance.sol</code>	Stores each wallet's current level (updated by the off-chain indexer), tallies level-weighted votes on asset/direction, enforces quorum (below quorum, previous decision stands). Holds no funds.
<code>WarchestVault.sol</code>	Holds treasury funds. Executes only pre-validated orders coming from governance (open position via keeper call, enforce per-trade cap e.g. 20% of treasury, enforce stop-loss). Holds no voting logic.
Indexer (off-chain service)	Runs daily snapshots, computes each wallet's holder-level and FIFO lot state from on-chain transfer history, pushes compact level updates to <code>WarchestGovernance</code> .
Keeper bot (off-chain service)	Reads vote outcomes from <code>WarchestGovernance</code> , bridges vault funds Robinhood Chain → Hyperliquid via Across, opens/closes positions via Hyperliquid's API using a trading-only agent wallet (no withdrawal rights), reports results back to the vault.

Security note: the keeper's Hyperliquid agent wallet must be scoped to trading only, never given withdrawal permission. This is the single most important operational safeguard in the whole system — verify this scoping is actually enforceable in Hyperliquid's agent-wallet permission model before writing a single line of keeper code.

3 Build Order and Rationale

1. **Token + Hook.** Lowest risk, best-documented pattern (Stakd is a live reference), and everything else depends on the fee actually landing in the vault correctly. Get this audited before moving on.
2. **Governance contract (levels + voting).** Can be built and tested independently of the vault and keeper — fake/mock treasury calls during this phase.
3. **Vault contract.** Wire it to governance once both are independently tested. Add the per-trade cap and stop-loss logic here, test exhaustively with mock keeper calls before touching real funds.
4. **Indexer.** Build once the governance contract's expected level-update interface is finalized.
5. **Keeper bot.** Build last, deliberately. It is the most complex and highest-risk component (cross-chain, external API, real capital at risk). Everything else should already be live and tested on testnet before this is attempted with real funds.

4 Pre-Mainnet Verification Checklist

- Confirm the hook correctly distinguishes buy vs. sell direction in all pool states (not just the simple case) on testnet.
- Confirm gas cost of the FIFO lot-consumption logic under a realistic worst case (wallet with many small buys) — stays cheap on Robinhood Chain's L2, but measure it, don't assume it.
- Confirm the indexer's snapshot-to-level update batches scale to expected holder counts without excessive on-chain write cost.
- Confirm Hyperliquid agent-wallet permission scoping (trade-only, no withdrawal) is actually enforced by Hyperliquid, not just configured client-side.
- Confirm Across bridging time and cost from Robinhood Chain to Hyperliquid under realistic treasury-sized transfers, not just small test amounts.
- Dry-run a full cycle end to end on testnet: vote → quorum check → vault order → bridge → Hyperliquid position → close → bridge back → distribution.
- Confirm the quorum fallback behavior (previous decision holding) works correctly when a vote fails to reach quorum.

5 Roadmap

1. Week 1: Token + Hook build, testnet deploy, begin audit conversation.
2. Week 2: Governance contract build + test (mocked vault).
3. Week 2–3: Vault contract build + test (mocked keeper), wire to governance.
4. Week 3: Indexer build, connect to governance on testnet.
5. Week 4: Keeper bot build, full testnet dry run of the entire cycle.
6. Week 5: Light audit pass on the full assembled system, fix findings.
7. Week 6: Mainnet launch, Phase 1 live, small initial treasury cap while monitoring real behavior.